APUNTES UNIVERSITARIOS

Foundations of Machine Learning and Deep Learning

Apuntes del Tema 5: Multilayer Perceptron (MLP I y II)

18 de octubre de 2026

Índice general

1 Apuntes del Tema 5: Multilayer Perceptron (MLP I y II) 2

1.1 1. De Modelos Lineales a No Lineales 2

1.1.1 1.1 Limitaciones de la Transformación Afín 2

1.2 2. Capas Ocultas y Funciones de Activación 2

1.2.1 2.1 Principales Funciones de Activación 2

1.3 3. Clasificación Multiclase: Regresión Softmax 3

1.3.1 3.1 Operador Softmax 3

1.3.2 3.2 Pérdida de Entropía Cruzada Categorical (*Categorical Cross-Entropy*) 3

1.4 4. Algoritmo de Propagación: Forward Pass y Backpropagation 3

1.4.1 4.1 Forward Pass (Propagación hacia Adelante) 3

1.4.2 4.2 Backpropagation (Propagación hacia Atrás) 4

1.5 5. Propiedades Teóricas e Inicialización de Parámetros 4

1.5.1 5.1 Teorema de Aproximación Universal 4

1.5.2 5.2 Rompimiento de la Simetría e Inicialización 4

1.6 6. Problemas en Redes Profundas: Desvanecimiento y Explosión del Gradiente . 5

1.7 7. Práctica y Aplicación en Código (PyTorch) 5

1 Apuntes del Tema 5: Multilayer Perceptron (MLP I y II)

1.1 1. De Modelos Lineales a No Lineales

1.1.1 1.1 Limitaciones de la Transformación Afín

- **Modelos Lineales:** Modelos como la Regresión Lineal o la Regresión Logística calculan transformaciones afines de la forma $z = x^\top w + b$.
- **El Problema del Colapso Lineal:** Si se apilan múltiples capas compuestas únicamente por transformaciones afines, la red colapsa matemáticamente en un único modelo lineal equivalente.
- *Demostración:* Dadas dos capas lineales $H = XW^{(1)} + b^{(1)}$ y $O = HW^{(2)} + b^{(2)}$:

$$O = (XW^{(1)} + b^{(1)})W^{(2)} + b^{(2)} = X(W^{(1)}W^{(2)}) + (b^{(1)}W^{(2)} + b^{(2)}) = XW + b$$

donde $W = W^{(1)}W^{(2)}$ y $b = b^{(1)}W^{(2)} + b^{(2)}$. La capacidad expresiva no aumenta con respecto a un modelo de una sola capa.

- **Separabilidad No Lineal (Problema XOR):** Las fronteras de decisión rectilíneas son incapaces de separar clases complejas o disjuntas (como la función lógica XOR o patrones circulares/concéntricos).

1.2 2. Capas Ocultas y Funciones de Activación

Para dotar al Perceptrón Multicapa (MLP) de la capacidad de aprender relaciones no lineales, se aplica una **función de activación no lineal** $\phi(\cdot)$ elemento a elemento tras cada transformación afín en las capas ocultas.

```

Entrada X (n x d)    [ Z = X·W(1) + b(1) ]    [ H =  (Z) ]    Salida O
```

1.2.1 2.1 Principales Funciones de Activación

Función	Ecuación Matemática $\phi(z)$	Rango de Salida	Propiedades y Derivada $\phi'(z)$
Sigmoide	$\sigma(z) = \frac{1}{1+e^{-z}}$	$(0, 1)$	Interpretación probabilística. **Problema:** Muestra zonas llanas en los extremos donde la derivada $\sigma'(z) = \sigma(z)(1-\sigma(z))$ tiende a 0, provocando desvanecimiento de gradiente (*vanishing gradient*).

|

| **Tanh** *(Tangente Hiperbólica)* | $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$ | $(-1, 1)$ | Centrada en cero (facilita la optimización). Sufre de desvanecimiento de gradientes para $|z|$ elevados ($\tanh'(z) = 1 - \tanh^2(z)$). |

| **ReLU** *(Rectified Linear Unit)* | $\text{ReLU}(z) = \max(0, z)$ | $[0, \infty)$ | **Estándar en redes profundas:** Computacionalmente eficiente. Derivada $\phi'(z) = 1$ para $z > 0$ (evita el desvanecimiento de gradientes en la región positiva). **Riesgo:** *Dying ReLU* si $z \leq 0$ de forma permanente. |

1.3 3. Clasificación Multiclase: Regresión Softmax

Cuando el target y pertenece a una de las K categorías posibles ($K > 2$) sin un orden explícito, se emplea la codificación **One-Hot Encoding** ($y \in \mathbb{R}^K$).

1.3.1 3.1 Operador Softmax

La capa de salida genera K valores no acotados denominados **logits** ($O = [o_1, o_2, \ldots, o_K]$). Para transformar los logits en una distribución de probabilidad bien definida donde $\sum_{k=1}^{K} \hat{y}_k = 1$ y $\hat{y}_k > 0$, se aplica la función **Softmax**:

$$\hat{y}_k = \text{Softmax}(o_k) = \frac{e^{o_k}}{\sum_{j=1}^{K} e^{o_j}}$$

- **Propiedad Conservadora del Orden:** Preserva la ordenación monótona de los logits ($\arg\max_k \hat{y}_k = \arg\max_k o_k$).

1.3.2 3.2 Pérdida de Entropía Cruzada Categorical (*Categorical Cross-Entropy*)

Para una muestra con vector One-Hot y y predicciones $\hat{y}$:

$$\mathcal{L}(y, \hat{y}) = -\sum_{k=1}^{K} y_k \log(\hat{y}_k)$$

> **Nota de Implementación (PyTorch):** En la práctica, no se añade manualmente la capa `Softmax` al final del modelo. Se retorna el tensor de logits directamente y se utiliza `nn.CrossEntropyLoss()`, la cual combina internamente `Softmax` y `Cross-Entropy` de forma numéricamente estable utilizando el truco *log-sum-exp*.

1.4 4. Algoritmo de Propagación: Forward Pass y Backpropagation

1.4.1 4.1 Forward Pass (Propagación hacia Adelante)

Para un MLP de L capas con entrada $X \in \mathbb{R}^{n \times d}$ ($H^{(0)} = X$):

Para $i = 1, 2, \ldots, L - 1$:

$$Z^{(i)} = H^{(i-1)} W^{(i)} + b^{(i)}$$

$$H^{(i)} = \phi(Z^{(i)})$$

Salida final (L):

$$O = H^{(L-1)}W^{(L)} + b^{(L)}$$

1.4.2 4.2 Backpropagation (Propagación hacia Atrás)

Consiste en la aplicación sistemática de la **Regla de la Cadena** multivariante del cálculo desde la función de pérdida $\mathcal{J}$ hasta las primeras capas de la red para obtener los gradientes $\frac{\partial \mathcal{J}}{\partial W^{(i)}}$ y $\frac{\partial \mathcal{J}}{\partial b^{(i)}}$:

$$\frac{\partial \mathcal{J}}{\partial W^{(i)}} = (H^{(i-1)})^\top \cdot \delta^{(i)}, \quad \text{donde } \delta^{(i)} = \frac{\partial \mathcal{J}}{\partial Z^{(i)}}$$

- **Gestión de Memoria:** Los tensores intermedios calculados durante el *forward pass* ($H^{(i)}, Z^{(i)}$) deben almacenarse en memoria para ser reevaluados durante el *backpropagation*, por lo que entrenar una red neuronal consume considerablemente más memoria que ejecutar inferencia (`with torch.no_grad():`).

1.5 5. Propiedades Teóricas e Inicialización de Parámetros

1.5.1 5.1 Teorema de Aproximación Universal

- **Definición:** Un Perceptrón Multicapa con tan solo **una capa oculta** y un número suficiente de neuronas no lineales puede aproximar cualquier función continua en un dominio compacto con un grado de precisión arbitrario $\epsilon > 0$.

- **¿Por qué usar redes profundas (más de 1 capa)?** El teorema demuestra la *existencia* teórica de dicha red, pero no especifica la cantidad de neuronas requeridas (que puede ser masiva/exponencial) ni si sus pesos se pueden aprender eficazmente con descenso de gradiente. Añadir profundidad permite representar funciones complejas mediante composiciones jerárquicas con un **número drásticamente menor de parámetros**.

1.5.2 5.2 Rompimiento de la Simetría e Inicialización

- **Inconveniente de Inicialización en Cero:** Si todos los pesos de una capa oculta se inicializan con la misma constante (p. ej. $w_{ij} = 0$), todas las neuronas de esa capa calcularán el mismo valor en el *forward pass* y recibirán exactamente el mismo gradiente en el *backward pass*. El MLP colapsaría al comportamiento de un modelo lineal único.

- **Estrategias de Inicialización:**

- **Xavier / Glorot Initialization:** Diseñada para activaciones sigmoideas o tanh. Mantiene constante la varianza de las activaciones y gradientes a través de las capas:

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right)$$

- **He / Kaiming Initialization:** Diseñada para activaciones **ReLU**, compensando que la mitad de las neuronas se desactivan (≤ 0):

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right)$$

1.6 6. Problemas en Redes Profundas: Desvanecimiento y Explosión del Gradiente

Al encadenar múltiples capas mediante la regla de la cadena, la magnitud del gradiente acumulado en las capas iniciales se calcula como un producto iterativo de matrices de pesos y derivadas de activación:

$$\frac{\partial \mathcal{J}}{\partial W^{(1)}} \propto \prod_{l=2}^{L} W^{(l)} \cdot \phi'(Z^{(l)})$$

- **Vanishing Gradients (Desvanecimiento):** Si las derivadas o los valores propios de los pesos son menores a 1 (p. ej., con sigmoide/tanh en zonas de saturación), el producto decrece exponencialmente hacia 0 conforme la red es más profunda. Las capas iniciales dejan de actualizarse y no aprenden.

- **Exploding Gradients (Explosión):** Si los valores de los pesos son mayores a 1, el producto crece exponencialmente provocando actualizaciones numéricamente inestables (`NaN` o `Inf`). Se mitiga aplicando **Gradient Clipping** (`nn.utils.clip_grad_norm_`).

1.7 7. Práctica y Aplicación en Código (PyTorch)

```
1
2  import torch
3
4  import torch.nn as nn
5
6  import torch.optim as optim
7
8
9
10 # ------------------------------------------------------------------------
11
12 # 1. Definición del Modelo MLP Multiclase
13
14 # ------------------------------------------------------------------------
15
16 class DeepMLP(nn.Module):
17
18     def __init__(self, input_dim=8, hidden_dims=[12, 6], num_classes=3, dropout_rate
   =0.2):
19
20         super(DeepMLP, self).__init__()
21
22
23
24         layers = []
25
26         in_dim = input_dim
27
28
29
```

```
        # Construcción de las capas ocultas con ReLU y Dropout

        for h_dim in hidden_dims:

            linear_layer = nn.Linear(in_dim, h_dim)

            # Inicialización He / Kaiming recomendada para ReLU

            nn.init.kaiming_normal_(linear_layer.weight, nonlinearity='relu')

            nn.init.zeros_(linear_layer.bias)


            layers.append(linear_layer)

            layers.append(nn.ReLU())

            layers.append(nn.Dropout(p=dropout_rate))

            in_dim = h_dim


        # Capa de salida lineal (entrega logits directos)

        out_layer = nn.Linear(in_dim, num_classes)

        nn.init.xavier_normal_(out_layer.weight)

        nn.init.zeros_(out_layer.bias)

        layers.append(out_layer)


        self.network = nn.Sequential(*layers)


    def forward(self, x):

        return self.network(x)


# ------------------------------------------------------------------------

# 2. Bucle de Entrenamiento

# ------------------------------------------------------------------------

def train_mlp_multiclass():

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")


```

```
    # Datos sintéticos: 100 muestras, 8 características, 3 clases

    X_dummy = torch.randn(100, 8)

    y_dummy = torch.randint(0, 3, (100,)) # Clases: 0, 1, 2


    model = DeepMLP(input_dim=8, hidden_dims=[12, 6], num_classes=3).to(device)


    # CrossEntropyLoss combina LogSoftmax + NLLLoss

    criterion = nn.CrossEntropyLoss()

    optimizer = optim.Adam(model.parameters(), lr=0.005)


    X_b = X_dummy.to(device)

    y_b = y_dummy.to(device)


    model.train()

    for epoch in range(1, 101):

        optimizer.zero_grad()


        # 1. Forward Pass (Logits)

        logits = model(X_b)

        loss = criterion(logits, y_b)


        # 2. Backward Pass

        loss.backward()


        # 3. Gradient Clipping opcional para evitar explosión

        nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)


        # 4. Actualización de parámetros

        optimizer.step()

```

```
146
147
148         if epoch % 25 == 0:
149
150             print(f"Época [{epoch:3d}/100] | Loss: {loss.item():.4f}")
151
152
153
154 if __name__ == "__main__":
155
156     train_mlp_multiclass()
```